

Programming Assignment 2 (Build a World, Then Learn It): Report

GitHub link to the repository:

<https://github.com/iterrall/cs272-26fa-pa2>

Rendered Map / State Diagram:

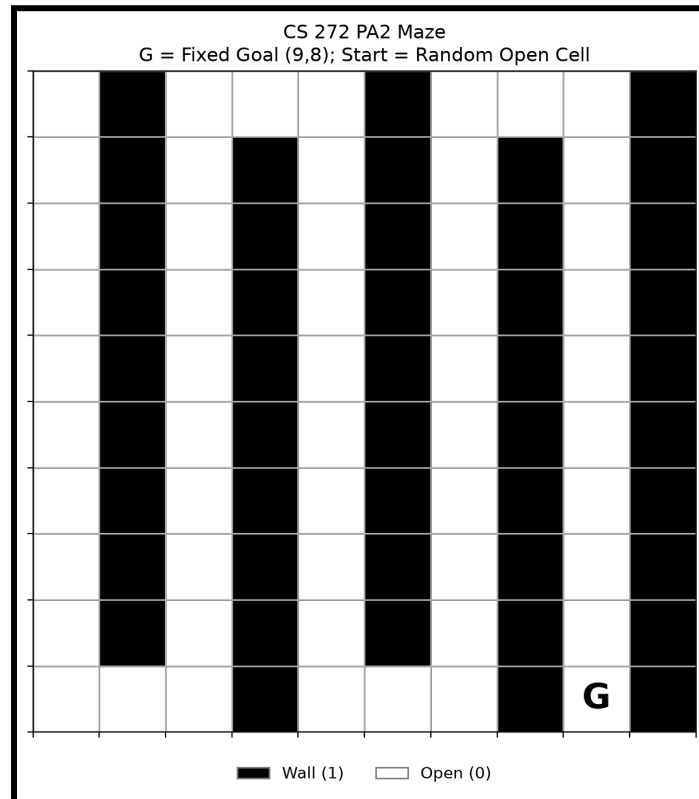


Figure 1. The 10x10 stochastic maze. 0 represents an open cell and 1 represents a wall. The goal is fixed at (9,8), while the starting position is randomly selected from the open cells.

Environment Description:

The environment is a stochastic 10x10 maze in which the agent must navigate from a randomly selected open cell to a fixed goal at position (9,8). The maze contains open cells represented by 0 and walls represented by 1. The agent can choose four actions: up, right, down, or left. An intended action is followed with probability 0.80, while with probability 0.20 the agent slips in one of the two perpendicular directions, with each slip direction having probability 0.10. If the resulting movement enters an open non-goal cell, the agent receives a reward of -0.10; attempting to move into a wall or outside the maze gives -0.35. Reaching the goal gives a reward of +10.0 and terminates the episode. The agent's state is encoded as $\text{row} * 10 + \text{column}$, giving

an observation space of Discrete(100). The environment itself never sets truncated=True; instead, the Gymnasium TimeLimit wrapper truncates an episode after 300 steps.

Sample Greedy-Policy Episode:

After training the SARSA(λ) agent for 5,000 episodes, one greedy-policy episode was generated using $\lambda = 0.9$ and seed 11. Because the starting position is randomized, this episode began at (0,7). The learned policy reached the fixed goal at (9,8) in 10 steps.

The complete step-by-step ANSI rendering is available in the repository at [results/sample_greedy_run.md](https://github.com/ajaynair/gymnasium/blob/main/results/sample_greedy_run.md). The initial and final states are shown below.

```
 $\lambda = 0.9$ 
Seed = 11
Initial state:

.#...#...#
A#.#.#.#.#
.#.#.#.#.#
.#.#.#.#.#
.#.#.#.#.#
.#.#.#.#.#
.#.#.#.#.#
.#.#.#.#.#
.#.#.#.#.#
...#...#G#
[ Legend: Agent (A) at (1, 0) | Goal (G) at (9, 8) ]
Step 1: action=2, reward=-0.1 → [ Legend: Agent (A) at (2, 0) | Goal (G) at (9, 8) ]
Step 2: action=2, reward=-0.1 → [ Legend: Agent (A) at (3, 0) | Goal (G) at (9, 8) ]
Step 3: action=2, reward=-0.35 → [ Legend: Agent (A) at (3, 0) | Goal (G) at (9, 8) ]
Step 4: action=2, reward=-0.35 → [ Legend: Agent (A) at (3, 0) | Goal (G) at (9, 8) ]
Step 5: action=2, reward=-0.35 → [ Legend: Agent (A) at (3, 0) | Goal (G) at (9, 8) ]
Step 6: action=2, reward=-0.35 → [ Legend: Agent (A) at (3, 0) | Goal (G) at (9, 8) ]
Step 7: action=2, reward=-0.1 → [ Legend: Agent (A) at (4, 0) | Goal (G) at (9, 8) ]
Step 8: action=2, reward=-0.1 → [ Legend: Agent (A) at (5, 0) | Goal (G) at (9, 8) ]
Step 9: action=2, reward=-0.1 → [ Legend: Agent (A) at (6, 0) | Goal (G) at (9, 8) ]
Step 10: action=2, reward=-0.1 → [ Legend: Agent (A) at (7, 0) | Goal (G) at (9, 8) ]
Step 11: action=2, reward=-0.1 → [ Legend: Agent (A) at (8, 0) | Goal (G) at (9, 8) ]
Step 12: action=2, reward=-0.35 → [ Legend: Agent (A) at (8, 0) | Goal (G) at (9, 8) ]
Step 13: action=2, reward=-0.1 → [ Legend: Agent (A) at (9, 0) | Goal (G) at (9, 8) ]
Step 14: action=1, reward=-0.1 → [ Legend: Agent (A) at (9, 1) | Goal (G) at (9, 8) ]
Step 15: action=1, reward=-0.1 → [ Legend: Agent (A) at (9, 2) | Goal (G) at (9, 8) ]
Step 16: action=0, reward=-0.1 → [ Legend: Agent (A) at (8, 2) | Goal (G) at (9, 8) ]
Step 17: action=0, reward=-0.1 → [ Legend: Agent (A) at (7, 2) | Goal (G) at (9, 8) ]
Step 18: action=0, reward=-0.1 → [ Legend: Agent (A) at (6, 2) | Goal (G) at (9, 8) ]
Step 19: action=0, reward=-0.1 → [ Legend: Agent (A) at (5, 2) | Goal (G) at (9, 8) ]
Step 20: action=0, reward=-0.1 → [ Legend: Agent (A) at (4, 2) | Goal (G) at (9, 8) ]
Step 21: action=0, reward=-0.1 → [ Legend: Agent (A) at (3, 2) | Goal (G) at (9, 8) ]
Step 22: action=0, reward=-0.1 → [ Legend: Agent (A) at (2, 2) | Goal (G) at (9, 8) ]
Step 23: action=0, reward=-0.1 → [ Legend: Agent (A) at (1, 2) | Goal (G) at (9, 8) ]
Step 24: action=0, reward=-0.1 → [ Legend: Agent (A) at (0, 2) | Goal (G) at (9, 8) ]
Step 25: action=1, reward=-0.1 → [ Legend: Agent (A) at (0, 3) | Goal (G) at (9, 8) ]
Step 26: action=1, reward=-0.1 → [ Legend: Agent (A) at (0, 4) | Goal (G) at (9, 8) ]
Step 27: action=2, reward=-0.1 → [ Legend: Agent (A) at (0, 3) | Goal (G) at (9, 8) ]
Step 28: action=1, reward=-0.1 → [ Legend: Agent (A) at (0, 4) | Goal (G) at (9, 8) ]
```


[Legend: Agent (A) at (9, 8) | Goal (G) at (9, 8)]

Here are the results:

- Lambda: 0.9
- Seed: 11
- Steps: 74
- Undiscounted return: -1.30
- Discounted return: -3.32
- Reached goal: True

λ Sweep Plot and Table:

Each SARSA(λ) agent was trained for 5,000 episodes. The λ sweep used in [0, 0.3, 0.6, 0.9, 1.0]. Five seeds were used for each λ : 11, 22, 33, 44, 55. We recorded returns for every training episode. Learning curves were smoothed over a 100-episode window. The target return used for the first-hit analysis was 7.0.

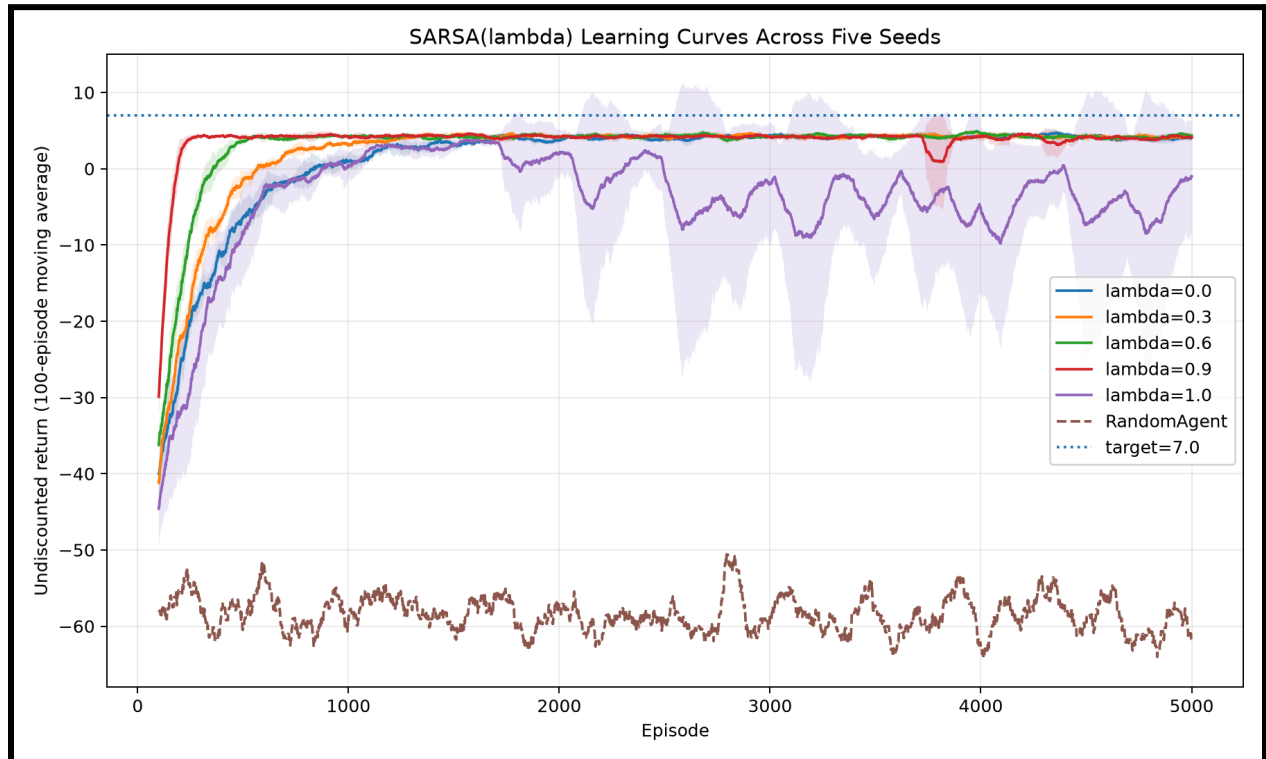


Figure 2. SARSA(λ) learning curves across five random seeds. Mean episode return is shown for $\lambda \in \{0, 0.3, 0.6, 0.9, 1.0\}$ over 5,000 training episodes. Curves are smoothed using a 100-episode window, and the shaded region represents variation across the five seeds.

λ	Mean episodes to first reach 7.0	Seeds reaching target	Mean final return (last 100)	Std. dev.
0.0	19.0	5/5	4.076	0.290
0.3	12.0	5/5	4.165	0.623

0.6	9.4	5/5	4.348	0.135
0.9	9.8	5/5	4.053	0.320
1.0	20.0	5/5	-1.001	7.299

Table 1. SARSA(λ) λ -sweep results across five random seeds. The target return threshold was set to 7.0. “Mean episodes to first reach 7.0” reports the mean episode number at which each seed first reached the threshold. “Mean final return” is the average return over the final 100 training episodes, and the standard deviation measures variation in those final-100 means across the five seeds.

The results show that all five λ values reached the target threshold in all five seeds. The mean episode at which the target was first reached ranged from 9.4 to 20.0 episodes. For λ values from 0.0 through 0.9, the mean return over the final 100 episodes ranged from 4.053 to 4.348. The $\lambda=1.0$ condition had a lower mean final return of -1.001 and substantially greater variation across seeds.

Why does λ do that here?

In this maze, the greater reward (+10.0) only shows up at the goal, after a long path with small penalties along the way. The question is how much of the final +10.0 should be tied to an earlier decision, like turning down a right turn at step 4, versus another step later on in the path. Eligibility traces handle this by marking every state-action pair that is visited, and letting the marks “fade” as more time passes. So when the +10.0 is achieved at the goal, the update flows backward through the path taken, with earlier steps getting a smaller share (controlled by lambda). When lambda = 0.0, only the last step before the reward gets updated, causing the agent to relearn the maze one step at a time across multiple episodes. Raising lambda to around 0.6 to 0.9 lets a single good episode update most of the path at once, which is why the earlier decisions are learned faster. However, with lambda at 1.0, this backfires as the maze also incorporates a random slip probability. A slip 30 steps in ends up being tied to a decision made way back at step 2. So these sweep results make sense: moderate lambda speeds up learning by spreading the reward across the path that earned it, but pushing lambda all the way to 1.0 spreads the noise from bad luck just as evenly. This is why performance fell apart, as shown by the table above.